



Practical Report - <05>
SE3082 - Parallel Computing
3rd year 1st semester - 2026

Student IDs: IT24104152

Exercise 01

```
ubuntu@ip-172-31-9-193:~/ex1$ gcc -fopenmp ex1.c -o ex1
ubuntu@ip-172-31-9-193:~/ex1$ ./ex1
thread # 0
thread # 1
Outside Barrier # 0
Hi again from thread # 0
Outside Barrier # 1

ubuntu@ip-172-31-9-193:~/ex1$ OMP_NUM_THREADS=4 ./ex1
thread # 2
thread # 0
thread # 3
thread # 1
Outside Barrier # 1
Hi again from thread # 1
Outside Barrier # 2
Hi again from thread # 2
Outside Barrier # 0
Hi again from thread # 0
Outside Barrier # 3
Hi again from thread # 3
ubuntu@ip-172-31-9-193:~/ex1$
```

In this program the `#pragma omp barrier` makes all the threads stop at the same point and wait for each other. First every thread prints its own thread number, then it reaches the barrier. No thread is allowed to go past the barrier until all the threads have reached it. That is why in the output all the "thread #" lines are printed first, and only after that the "Outside Barrier" and "Hi again" lines are printed. The order inside each group is random because the threads run at the same time, but the two groups never mix together because of the barrier.

Exercise 02

```
ubuntu@ip-172-31-9-193:~/ex2$ gcc -fopenmp ex2.c -o ex2
ubuntu@ip-172-31-9-193:~/ex2$ ./ex2
thread # 0
Inside Single # 0
Exiting Single
thread # 1
Hi again from thread # 0
Hi again from thread # 1
ubuntu@ip-172-31-9-193:~/ex2$ OMP_NUM_THREADS=4 ./ex2
thread # 3
thread # 2
thread # 1
Inside Single # 1
Exiting Single
thread # 0
Hi again from thread # 1
Hi again from thread # 0
Hi again from thread # 2
Hi again from thread # 3
ubuntu@ip-172-31-9-193:~/ex2$ |
```

The `#pragma omp single` makes only one thread run the code inside the block, not all of them. In the output the "Inside Single" line is printed only once even though there are 4 threads. The thread that runs it is not fixed, it is just whichever thread reaches it first, so the thread number can change each time I run it. The single also has a hidden barrier at the end, so the other threads wait until the single block is finished before they print "Hi again". That is why all the "Hi again" lines come after the single block.

Exercise 03

```
ubuntu@ip-172-31-9-193:~/ex3$ gcc -fopenmp ex3.c -o ex3
ubuntu@ip-172-31-9-193:~/ex3$ ./ex3
thread # 0
Inside Master # 0
Exiting Master
Hi again from thread # 0
thread # 1
Hi again from thread # 1

ubuntu@ip-172-31-9-193:~/ex3$ OMP_NUM_THREADS=4 ./ex3
thread # 3
Hi again from thread # 3
thread # 1
Hi again from thread # 1
thread # 2
Hi again from thread # 2
thread # 0
Inside Master # 0
Exiting Master
Hi again from thread # 0
ubuntu@ip-172-31-9-193:~/ex3$
```

The `#pragma omp master` makes only the master thread (thread 0) run the block, so "Inside Master # 0" is always printed by thread 0 every time I run it. This is different from single, where any thread could run it. The other main difference is that master does not have a barrier at the end. The other threads do not wait for thread 0, they just skip the master block and keep going. Because of this the "Hi again" lines can appear before or after the master block, the order is not guaranteed.

Exercise 04

1) With “#pragma omp parallel for”

```
ubuntu@ip-172-31-9-193:~/ex4$ ./ex4
r = 1, k = 1, thread # = 0
r = 1, k = 2, thread # = 0
r = 1, k = 3, thread # = 0
r = 1, k = 4, thread # = 0
r = 1, k = 5, thread # = 0
r = 1, k = 6, thread # = 0
r = 1, k = 7, thread # = 0
r = 1, k = 8, thread # = 0
r = 1, k = 9, thread # = 0
r = 1, k = 10, thread # = 0
r = 2, k = 1, thread # = 0
r = 2, k = 2, thread # = 0
r = 2, k = 3, thread # = 0
r = 2, k = 4, thread # = 0
r = 2, k = 5, thread # = 0
r = 2, k = 6, thread # = 0
r = 2, k = 7, thread # = 0
r = 2, k = 8, thread # = 0
r = 2, k = 9, thread # = 0
r = 2, k = 10, thread # = 0
r = 3, k = 1, thread # = 0
r = 3, k = 2, thread # = 0
r = 3, k = 3, thread # = 0
r = 3, k = 4, thread # = 0
r = 3, k = 5, thread # = 0
r = 3, k = 6, thread # = 0
r = 3, k = 7, thread # = 0
r = 3, k = 8, thread # = 0
r = 3, k = 9, thread # = 0
r = 3, k = 10, thread # = 0
r = 4, k = 1, thread # = 1
r = 4, k = 2, thread # = 1
r = 4, k = 3, thread # = 1
r = 4, k = 4, thread # = 1
r = 4, k = 5, thread # = 1
r = 4, k = 6, thread # = 1
r = 4, k = 7, thread # = 1
r = 4, k = 8, thread # = 1
r = 4, k = 9, thread # = 1
r = 4, k = 10, thread # = 1
r = 5, k = 1, thread # = 1
r = 5, k = 2, thread # = 1
r = 5, k = 3, thread # = 1
r = 5, k = 4, thread # = 1
r = 5, k = 5, thread # = 1
r = 5, k = 6, thread # = 1
r = 5, k = 7, thread # = 1
r = 5, k = 8, thread # = 1
r = 5, k = 9, thread # = 1
r = 5, k = 10, thread # = 1
ubuntu@ip-172-31-9-193:~/ex4$
```

```
ubuntu@ip-172-31-9-193: ~/ex4
ubuntu@ip-172-31-9-193:~/ex4$ OMP_NUM_THREADS=4 ./ex4
r = 5, k = 1, thread # = 3
r = 5, k = 2, thread # = 3
r = 5, k = 3, thread # = 3
r = 5, k = 4, thread # = 3
r = 5, k = 5, thread # = 3
r = 5, k = 6, thread # = 3
r = 5, k = 7, thread # = 3
r = 5, k = 8, thread # = 3
r = 5, k = 9, thread # = 3
r = 5, k = 10, thread # = 3
r = 3, k = 1, thread # = 1
r = 3, k = 2, thread # = 1
r = 3, k = 3, thread # = 1
r = 3, k = 4, thread # = 1
r = 3, k = 5, thread # = 1
r = 3, k = 6, thread # = 1
r = 3, k = 7, thread # = 1
r = 3, k = 8, thread # = 1
r = 3, k = 9, thread # = 1
r = 3, k = 10, thread # = 1
r = 1, k = 1, thread # = 0
r = 1, k = 2, thread # = 0
r = 1, k = 3, thread # = 0
r = 1, k = 4, thread # = 0
r = 1, k = 5, thread # = 0
r = 1, k = 6, thread # = 0
r = 1, k = 7, thread # = 0
r = 1, k = 8, thread # = 0
r = 1, k = 9, thread # = 0
r = 1, k = 10, thread # = 0
r = 2, k = 1, thread # = 0
r = 2, k = 2, thread # = 0
r = 2, k = 3, thread # = 0
r = 2, k = 4, thread # = 0
r = 2, k = 5, thread # = 0
r = 2, k = 6, thread # = 0
r = 2, k = 7, thread # = 0
r = 2, k = 8, thread # = 0
r = 2, k = 9, thread # = 0
r = 2, k = 10, thread # = 0
r = 4, k = 1, thread # = 2
r = 4, k = 2, thread # = 2
r = 4, k = 3, thread # = 2
r = 4, k = 4, thread # = 2
r = 4, k = 5, thread # = 2
r = 4, k = 6, thread # = 2
r = 4, k = 7, thread # = 2
r = 4, k = 8, thread # = 2
r = 4, k = 9, thread # = 2
r = 4, k = 10, thread # = 2
ubuntu@ip-172-31-9-193:~/ex4$
```

Here #pragma omp parallel for only parallelizes the outer loop (the r loop). The inner k loop runs fully inside one thread. So each thread gets a whole r value together with all its 10 k iterations. Since there are 5 r values and 4 threads the work cannot divide evenly, one thread gets 2 r values and the others get 1 each. In the output all 10 k lines for the same r have the same thread number, which shows that one r row is handled by only one thread.

2) With “#pragma omp parallel for collapse(2)”

```
ubuntu@ip-172-31-9-193:~/ex4$ gcc -fopenmp ex4.c -o ex4
ubuntu@ip-172-31-9-193:~/ex4$ ./ex4
r = 1, k = 1, thread # = 0
r = 1, k = 2, thread # = 0
r = 1, k = 3, thread # = 0
r = 1, k = 4, thread # = 0
r = 1, k = 5, thread # = 0
r = 1, k = 6, thread # = 0
r = 1, k = 7, thread # = 0
r = 1, k = 8, thread # = 0
r = 1, k = 9, thread # = 0
r = 1, k = 10, thread # = 0
r = 2, k = 1, thread # = 0
r = 2, k = 2, thread # = 0
r = 2, k = 3, thread # = 0
r = 2, k = 4, thread # = 0
r = 2, k = 5, thread # = 0
r = 2, k = 6, thread # = 0
r = 2, k = 7, thread # = 0
r = 2, k = 8, thread # = 0
r = 2, k = 9, thread # = 0
r = 2, k = 10, thread # = 0
r = 3, k = 1, thread # = 0
r = 3, k = 2, thread # = 0
r = 3, k = 3, thread # = 0
r = 3, k = 4, thread # = 0
r = 3, k = 5, thread # = 0
r = 3, k = 6, thread # = 1
r = 3, k = 7, thread # = 1
r = 3, k = 8, thread # = 1
r = 3, k = 9, thread # = 1
r = 3, k = 10, thread # = 1
r = 4, k = 1, thread # = 1
r = 4, k = 2, thread # = 1
r = 4, k = 3, thread # = 1
r = 4, k = 4, thread # = 1
r = 4, k = 5, thread # = 1
r = 4, k = 6, thread # = 1
r = 4, k = 7, thread # = 1
r = 4, k = 8, thread # = 1
r = 4, k = 9, thread # = 1
r = 4, k = 10, thread # = 1
r = 5, k = 1, thread # = 1
r = 5, k = 2, thread # = 1
r = 5, k = 3, thread # = 1
r = 5, k = 4, thread # = 1
r = 5, k = 5, thread # = 1
r = 5, k = 6, thread # = 1
r = 5, k = 7, thread # = 1
r = 5, k = 8, thread # = 1
r = 5, k = 9, thread # = 1
r = 5, k = 10, thread # = 1
ubuntu@ip-172-31-9-193:~/ex4$
```

```
ubuntu@ip-172-31-9-193:~/ex4$ OMP_NUM_THREADS=4 ./ex4
r = 4, k = 9, thread # = 3
r = 4, k = 10, thread # = 3
r = 5, k = 1, thread # = 3
r = 5, k = 2, thread # = 3
r = 5, k = 3, thread # = 3
r = 5, k = 4, thread # = 3
r = 5, k = 5, thread # = 3
r = 5, k = 6, thread # = 3
r = 5, k = 7, thread # = 3
r = 5, k = 8, thread # = 3
r = 5, k = 9, thread # = 3
r = 5, k = 10, thread # = 3
r = 2, k = 4, thread # = 1
r = 2, k = 5, thread # = 1
r = 2, k = 6, thread # = 1
r = 2, k = 7, thread # = 1
r = 2, k = 8, thread # = 1
r = 2, k = 9, thread # = 1
r = 2, k = 10, thread # = 1
r = 3, k = 1, thread # = 1
r = 3, k = 2, thread # = 1
r = 3, k = 3, thread # = 1
r = 3, k = 4, thread # = 1
r = 3, k = 5, thread # = 1
r = 3, k = 6, thread # = 1
r = 1, k = 1, thread # = 0
r = 1, k = 2, thread # = 0
r = 1, k = 3, thread # = 0
r = 1, k = 4, thread # = 0
r = 1, k = 5, thread # = 0
r = 1, k = 6, thread # = 0
r = 1, k = 7, thread # = 0
r = 1, k = 8, thread # = 0
r = 1, k = 9, thread # = 0
r = 1, k = 10, thread # = 0
r = 2, k = 1, thread # = 0
r = 2, k = 2, thread # = 0
r = 2, k = 3, thread # = 0
r = 3, k = 7, thread # = 2
r = 3, k = 8, thread # = 2
r = 3, k = 9, thread # = 2
r = 3, k = 10, thread # = 2
r = 4, k = 1, thread # = 2
r = 4, k = 2, thread # = 2
r = 4, k = 3, thread # = 2
r = 4, k = 4, thread # = 2
r = 4, k = 5, thread # = 2
r = 4, k = 6, thread # = 2
r = 4, k = 7, thread # = 2
r = 4, k = 8, thread # = 2
ubuntu@ip-172-31-9-193:~/ex4$
```

When I use collapse(2) the two loops are joined into one big loop of $5 \times 10 = 50$ iterations, and these 50 iterations are shared among the threads. Now the unit of work is a single (r,k) pair instead of a whole r row. So in the output the same r value is now printed by different thread numbers (for example r=1 is split between two threads), which did not happen before. The work is more balanced because it is divided into smaller pieces.

Exercise 05

```
ubuntu@ip-172-31-9-193:~/ex5$ cat ex5.c
#include <omp.h>
#include <stdio.h>
#include <stdlib.h>

#define CUTOFF 20

long fib_serial(int n) {
    if (n < 2)
        return n;
    return fib_serial(n - 1) + fib_serial(n - 2);
}

long fib_parallel(int n) {
    if (n < 2)
        return n;
    if (n < CUTOFF)
        return fib_serial(n);

    long i, j;

    #pragma omp task shared(i)
    i = fib_parallel(n - 1);

    #pragma omp task shared(j)
    j = fib_parallel(n - 2);

    #pragma omp taskwait
    return i + j;
}

int main(int argc, char *argv[]) {
    int n = (argc > 1) ? atoi(argv[1]) : 40;
    double t0, t1;
    long result;

    t0 = omp_get_wtime();
    result = fib_serial(n);
    t1 = omp_get_wtime();
    printf("Serial : fib(%d) = %ld   time = %f s\n", n, result, t1 - t0);

    t0 = omp_get_wtime();
    #pragma omp parallel
    {
        #pragma omp single
        result = fib_parallel(n);
    }
    t1 = omp_get_wtime();
    printf("Parallel: fib(%d) = %ld   time = %f s   (threads = %d)\n",
        n, result, t1 - t0, omp_get_max_threads());

    return 0;
}
ubuntu@ip-172-31-9-193:~/ex5$
```

```
ubuntu@ip-172-31-9-193: ~/ex5
ubuntu@ip-172-31-9-193:~/ex5$ gcc -fopenmp ex5.c -o ex5
ubuntu@ip-172-31-9-193:~/ex5$ OMP_NUM_THREADS=1 ./ex5
Serial : fib(40) = 102334155   time = 0.938606 s
Parallel: fib(40) = 102334155   time = 0.962959 s   (threads = 1)
ubuntu@ip-172-31-9-193:~/ex5$ OMP_NUM_THREADS=2 ./ex5
Serial : fib(40) = 102334155   time = 0.934724 s
Parallel: fib(40) = 102334155   time = 0.772609 s   (threads = 2)
ubuntu@ip-172-31-9-193:~/ex5$ OMP_NUM_THREADS=4 ./ex5
Serial : fib(40) = 102334155   time = 0.969984 s
Parallel: fib(40) = 102334155   time = 0.733412 s   (threads = 4)
ubuntu@ip-172-31-9-193:~/ex5$ OMP_NUM_THREADS=8 ./ex5
Serial : fib(40) = 102334155   time = 0.931360 s
Parallel: fib(40) = 102334155   time = 0.727293 s   (threads = 8)
ubuntu@ip-172-31-9-193:~/ex5$ OMP_NUM_THREADS=16 ./ex5
Serial : fib(40) = 102334155   time = 0.932473 s
Parallel: fib(40) = 102334155   time = 0.722604 s   (threads = 16)
ubuntu@ip-172-31-9-193:~/ex5$
```

I parallelized the fibonacci using OpenMP tasks. Each recursive call $\text{fib}(n-1)$ and $\text{fib}(n-2)$ is made into a task with `#pragma omp task`, and `shared` is used so the results i and j can be read back by the parent. `#pragma omp taskwait` makes the parent wait until both tasks are finished before doing $i+j$. The whole thing is started inside a parallel region with `single`, so only one thread creates the first task and the other threads pick up the tasks that are spawned. I also added a cutoff value, because if I make a task for every single call it becomes much slower than serial due to the task creation overhead. Below the cutoff the code just runs serially so the tasks stay big enough to be worth it. The parallel answer is the same as the serial answer, which shows it is correct.

Exercise 06

```
ubuntu@ip-172-31-9-193:~/ex6$ cat ex6.c
#include <omp.h>
#include <stdio.h>
#include <stdlib.h>

#define N 1000000
#define STRIP 1024

static float A[N], B[N], C[N];

int main() {
    for (int i = 0; i < N; i++) {
        A[i] = (float)(i % 100);
        B[i] = (float)((i % 50) + 1);
    }

    double t0 = omp_get_wtime();

    #pragma omp parallel for schedule(static)
    for (int s = 0; s < N; s += STRIP) {
        int end = (s + STRIP < N) ? s + STRIP : N;
        #pragma omp simd
        for (int i = s; i < end; i++)
            C[i] = A[i] * B[i];
    }

    double t1 = omp_get_wtime();

    long checksum = 0;
    for (int i = 0; i < N; i++)
        checksum += (long)C[i];

    printf("N = %d, strip = %d, threads = %d\n", N, STRIP, omp_get_max_threads());
    printf("checksum = %ld\n", checksum);
    printf("C[0]=%.1f C[1]=%.1f C[N-1]=%.1f\n", C[0], C[1], C[N-1]);
    printf("time = %f s\n", t1 - t0);

    return 0;
}
ubuntu@ip-172-31-9-193:~/ex6$ |
```

```
ubuntu@ip-172-31-9-193:~/ex6$ gcc -fopenmp -O2 ex6.c -o ex6
ubuntu@ip-172-31-9-193:~/ex6$ OMP_NUM_THREADS=1 ./ex6
N = 1000000, strip = 1024, threads = 1
checksum = 1470500000
C[0]=0.0 C[1]=2.0 C[N-1]=4950.0
time = 0.002929 s
ubuntu@ip-172-31-9-193:~/ex6$ OMP_NUM_THREADS=2 ./ex6
N = 1000000, strip = 1024, threads = 2
checksum = 1470500000
C[0]=0.0 C[1]=2.0 C[N-1]=4950.0
time = 0.002453 s
ubuntu@ip-172-31-9-193:~/ex6$ OMP_NUM_THREADS=4 ./ex6
N = 1000000, strip = 1024, threads = 4
checksum = 1470500000
C[0]=0.0 C[1]=2.0 C[N-1]=4950.0
time = 0.002630 s
ubuntu@ip-172-31-9-193:~/ex6$ OMP_NUM_THREADS=8 ./ex6
N = 1000000, strip = 1024, threads = 8
checksum = 1470500000
C[0]=0.0 C[1]=2.0 C[N-1]=4950.0
time = 0.002884 s
ubuntu@ip-172-31-9-193:~/ex6$ OMP_NUM_THREADS=16 ./ex6
N = 1000000, strip = 1024, threads = 16
checksum = 1470500000
C[0]=0.0 C[1]=2.0 C[N-1]=4950.0
time = 0.003319 s
ubuntu@ip-172-31-9-193:~/ex6$ |
```

For this exercise I used strip mining. The single loop is broken into strips of a fixed size (I used 1024). The outer loop jumps by the strip size and the inner loop does the actual multiplication inside one strip. I parallelized the outer strip loop with `#pragma omp parallel` for so the strips are shared among the threads, and I put `#pragma omp simd` on the inner loop so it can use SIMD vector instructions. I chose 1024 as the strip size because it is a multiple of the SIMD vector width (8 floats for AVX2 and 16 for AVX-512), so each strip fits nicely into full vector registers with nothing left over. I compiled with `-O2` so the `simd` actually takes effect. The checksum is the same for different thread counts, which shows the result is correct.

Exercise 07

```
ubuntu@ip-172-31-9-193:~/ex7$ cat ex7.c
#include <stdio.h>
#include <stdlib.h>
#include <math.h>
#include <omp.h>

#define NPOINTS 2000
#define MAXITER 2000

struct complex{
    double real;
    double imag;
};

int main(){
    int i, j, iter, numoutside = 0;
    double area, error, ztemp;
    struct complex z, c;
    double tstart, tstop;

    /*
     * Outer loops run over npoints, initialise z=c
     * Inner loop has the iteration z=z*z+c, and threshold test
     */

    tstart = omp_get_wtime();

#pragma omp parallel for default(none) \
    private(i, j, iter, z, c, ztemp) \
    reduction(+:numoutside) schedule(static)
    for (i=0; i<NPOINTS; i++) {
        for (j=0; j<NPOINTS; j++) {
            c.real = -2.0+2.5*(double)(i)/(double)(NPOINTS)+1.0e-7;
            c.imag = 1.125*(double)(j)/(double)(NPOINTS)+1.0e-7;
            z=c;
            for (iter=0; iter<MAXITER; iter++){
                ztemp=(z.real*z.real)-(z.imag*z.imag)+c.real;
                z.imag=z.real*z.imag*2+c.imag;
                z.real=ztemp;
                if ((z.real*z.real+z.imag*z.imag)>4.0e0) {
                    numoutside++;
                    break;
                }
            }
        }
    }

    tstop = omp_get_wtime();

    /*
     * Calculate area and error and output the results
     */

    area=2.0*2.5*1.125*(double)(NPOINTS*NPOINTS-numoutside)/(double)(NPOINTS*NPOINTS);
    error=area/(double)NPOINTS;

    printf("Threads used = %d\n", omp_get_max_threads());
    printf("numoutside = %d\n", numoutside);
    printf("Area of Mandelbrot set = %12.8f +/- %12.8f\n", area, error);
    printf("Time = %f s\n", tstop - tstart);

    return 0;
}
ubuntu@ip-172-31-9-193:~/ex7$ |
```

```
ubuntu@ip-172-31-9-193:~/ex7$ OMP_NUM_THREADS=1 && ./ex7
Threads used = 2
numoutside = 2926744
Area of Mandlebrot set = 1.50926625 +/- 0.00075463
Time = 6.497757 s
ubuntu@ip-172-31-9-193:~/ex7$ OMP_NUM_THREADS=2 && ./ex7
Threads used = 2
numoutside = 2926744
Area of Mandlebrot set = 1.50926625 +/- 0.00075463
Time = 6.469723 s
ubuntu@ip-172-31-9-193:~/ex7$ OMP_NUM_THREADS=3 && ./ex7
Threads used = 2
numoutside = 2926744
Area of Mandlebrot set = 1.50926625 +/- 0.00075463
Time = 6.509353 s
ubuntu@ip-172-31-9-193:~/ex7$ OMP_NUM_THREADS=4 && ./ex7
Threads used = 2
numoutside = 2926744
Area of Mandlebrot set = 1.50926625 +/- 0.00075463
Time = 6.467765 s
ubuntu@ip-172-31-9-193:~/ex7$
```

I parallelized the outer loop with `#pragma omp parallel for`. The variables `i`, `j`, `iter`, `z`, `c` and `ztemp` are made private because each thread needs its own copy while it is iterating, otherwise the threads would overwrite each other. `numoutside` is a reduction variable with `reduction(+:numoutside)`, so each thread counts its own points outside the set and OpenMP adds them all together at the end, which avoids a data race. I ran it with 1, 2, 3 and 4 threads and the area and `numoutside` came out exactly the same in every case, which shows the parallel version is correct. On a multi core machine the time also goes down as more threads are added.